

VACF Transport Specification

GK1 after H0: Projected Green-Kubo Self-Diffusion

mdstats

2026-07-22

Contents

1 Purpose and status	1
2 Borrowed theory and numerical machinery	1
3 Scope	2
4 Public function	2
4.1 Subspace resolution	2
4.2 Weighting	3
4.3 Projection	3
4.4 Truncation	3
5 Result contract	3
6 Algorithm	4
7 Provenance	4
8 Required tests	4
9 References	4

1 Purpose and status

This document specifies the H0-hardened implementation in

`mdstats/analysis/vacf_transport.py`

for the public function

`integrate_vacf_to_diffusion(...)`

and the immutable `VACFDiffusionResult`. The implementation consumes an existing self `VACFResult`; it does not recompute velocities or correlations.

The H0 revision corrects the former ambiguous `dimensions=1/2` behavior. A dimensional divisor is now derived only from an explicit physical subspace. Full three-dimensional scalar calls and single Cartesian component calls retain their previous values.

2 Borrowed theory and numerical machinery

The Green-Kubo self-diffusion relation follows Green [1] and Kubo [2]. For an orthonormal subspace basis $B \in \mathbb{R}^{d \times 3}$,

$$C_B(t) = \text{tr} [BC(t)B^\top],$$

and

$$D_B(t) = \frac{1}{d} \int_0^t C_B(\tau) d\tau.$$

The sampled cumulative integral uses the composite trapezoidal rule through the shared `_quadrature.py` wrapper around SciPy [3]. These relations and the quadrature rule are borrowed. Explicit subspace selection, fail-closed tensor requirements, result provenance, and compatibility adapters are mdstats design.

3 Scope

The function provides:

- explicit Cartesian-axis or general orthonormal-subspace projection;
- a physical equal-per-particle self-VACF normalization;
- cumulative trapezoidal integration on stored lag coordinates;
- optional truncation at an existing lag boundary;
- canonical output in Angstrom squared per picosecond;
- a derived cm^2/s view;
- complete projection and input-signature provenance; and
- deeply immutable arrays and metadata.

It does not provide plateau discovery, tail fitting, uncertainty estimation, collective transport, conductivity, or Nernst-Einstein comparison.

4 Public function

```
def integrate_vacf_to_diffusion(
    vacf: VACFResult,
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    dimensions: Literal[1, 2, 3] | None = None,
    component: Literal["scalar", "x", "y", "z"] = "scalar",
    maximum_time_ps: float | None = None,
    integration: Literal["trapezoid"] = "trapezoid",
) -> VACFDiffusionResult:
    ...
```

4.1 Subspace resolution

`axes` and `projection_basis` follow the shared H0 contract in `_dynamics_common_spec.md`.

The legacy arguments are compatibility adapters only:

- no subspace and `component="scalar"` selects full 3D;
- `component="x", "y", or "z"` selects that one Cartesian axis;
- scalar `dimensions=3` is accepted as a consistency check;
- scalar `dimensions=1` or `2` without `axes` or `projection_basis` raises;
- an explicit subspace and `dimensions` must have the same rank; and
- a non-scalar component cannot be combined with an explicit subspace.

Changing only a divisor may never reinterpret a full scalar result as a lower- dimensional observable.

4.2 Weighting

Physical self diffusion requires equal positive per-atom weights.

Accepted source cases are:

- `weighting="uniform"` with every stored weight equal to one; or
- `weighting="explicit"` with every stored weight equal to the same positive value.

Mass weighting and nonuniform explicit weights raise. The projected physical VACF is divided by `vacf.weight_sum` before the dimensional divisor is applied.

4.3 Projection

For a canonical axis subset, the implementation sums the selected stored Cartesian diagonal components. For a general basis, it computes

$$C_B(t) = \sum_{a=1}^d \mathbf{b}_a^\top C(t) \mathbf{b}_a.$$

A general rotated basis requires `vacf.tensor_sum`. If it is absent, the function raises rather than neglecting off-diagonal terms.

4.4 Truncation

`maximum_time_ps=None` retains all stored lags. Otherwise, the function retains the largest stored lag not exceeding the finite nonnegative request, with a small floating-point boundary tolerance. It does not interpolate a new endpoint.

5 Result contract

```
@dataclass(frozen=True, slots=True)
class VACFDiffusionResult:
    lag_times: NDArray[np.float64]
    running_diffusion_a2_per_ps: NDArray[np.float64]
    integrand: NDArray[np.float64]
    dimensions: int
    component: str
    weighting: str
    integration: str
    metadata: Mapping[str, Any]
    projection_basis: NDArray[np.float64]
    projection_labels: tuple[str, ...] | None
    signature: DynamicsInputSignature | None
```

Required identities are:

- all three numerical arrays have shape (L,) and finite values;
- lag time begins at zero and is strictly increasing;
- running diffusion begins at zero;
- `dimensions == projection_basis.shape[0]`;
- the basis rows are orthonormal;
- a single labeled axis agrees with `component`;
- the signature subspace, when present, equals the result subspace; and
- `running_diffusion_a2_per_ps` equals the cumulative trapezoidal integral of `integrand` on `lag_times`.

Every array is owned and read-only. Metadata is recursively immutable.

The derived property

result.running_diffusion_cm2_per_s
uses

$$1 \text{ Angstrom}^2/\text{ps} = 10^{-4} \text{ cm}^2/\text{s}.$$

6 Algorithm

1. Validate result type and integration method.
2. Resolve the physical subspace.
3. Validate lag coordinates and self-diffusion weighting.
4. Resolve the retained lag prefix.
5. project the stored VACF onto the selected subspace.
6. Divide the projected physical VACF by the subspace rank.
7. Apply cumulative trapezoidal integration.
8. Propagate the source signature with the resolved subspace.
9. Build immutable result arrays and provenance.

7 Provenance

Metadata records at least:

- source and resolved weighting;
- measured atom indices and weight sum;
- drift mode and drift-reference atoms;
- source backend, lag steps, and origin counts;
- requested and actual maximum time;
- basis, labels, rank, and normalization divisor;
- units and conversion factor;
- Green-Kubo references;
- quadrature method; and
- the complete source VACF metadata.

The final stored value is not declared converged and no plateau is selected.

8 Required tests

- full-3D scalar values agree with the pre-H0 reference;
- x, y, and z component values agree with the pre-H0 reference;
- anisotropic **xy** and **xz** projections select before dividing;
- a rotated basis uses off-diagonal tensor terms;
- a rotated basis without a tensor raises;
- scalar **dimensions=1/2** without a subspace raises;
- explicit subspace/rank inconsistencies raise;
- mass and nonuniform weights raise;
- cumulative integration agrees with a direct trapezoid oracle;
- truncation retains only existing lags;
- signature projection is propagated; and
- result arrays and nested metadata are immutable.

9 References

- [1] M. S. Green, *J. Chem. Phys.* **22**, 398-413 (1954). DOI: 10.1063/1.1740082.
[2] R. Kubo, *J. Phys. Soc. Jpn.* **12**, 570-586 (1957). DOI: 10.1143/JPSJ.12.570.

[3] P. Virtanen et al., *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.